



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC3413 — Implementación de Sistemas de Base de Datos — 1' 2026

LABORATORIO 3

Laboratorio: Operadores físicos e implementación de joins.
Publicación: Martes 6 de mayo.
Ayudantías: Viernes 8 de mayo.
Entrega: **Viernes 15 de mayo hasta las 23:59 horas (cupones valen hasta el lunes 18).**

Descripción del laboratorio

En este laboratorio exploraremos cómo IIC3413DB [1] procesa consultas SQL. Siguiendo la estructura de un motor SQL explicada en las clases, IIC3413DB usa tres componentes para procesar las consultas (la implementación se encuentra en la carpeta `query` del código):

- **parser**, cual contiene la definición de la gramática de un fragmento de SQL, parser de consultas que cumplan con esta gramática, el pre-processor cual genera el primer plan lógico, y un query rewriter cual permite mejorar el plan inicial.
- **optimizer**, cual se encarga de elegir el orden de joins y generar un plan físico; y
- **executor**, cual contiene la implementación de operadores físicos particulares (e.g. algoritmos para el join, la selección, etc.).

En este laboratorio ustedes solo trabajarán con el **executor**, pero para tener una idea del flujo de consultas en un motor relacional, a continuación explicaremos algunos detalles de cada componente.

Parser y plan lógico [informativo – no crucial para Lab3]

La componente **parser** del IIC3413DB se encarga de parsear consultas y generar el plan lógico. La implementación se encuentra en la carpeta `/src/query/parser`. El archivo `parser.h` contiene el flujo de esta componente de nuestro sistema. La secuencia de operaciones es la siguiente:

1. **Gramática, lexer y parser (carpeta grammar)**. Para automatizar la tarea de parsing de consultas y su análisis léxico, IIC3413DB ocupa la librería/herramienta ANTLR [2]. ANTLR permite definir los tokens léxicos de nuestro lenguaje (ver `IIC3413Lexer.g4` en la carpeta `/src/query/parser/grammar/`) y una gramática libre de contexto para las consultas permitidas (`IIC3413Parser.g4` en la misma carpeta)¹. Con estos dos archivos, y ejecutando el script `generate.sh`, ANTLR genera todo lo necesario para armar árbol de parseo para nuestras consultas. (Cómo referencia, este árbol es creado por el archivo `parser.h` en el comando `tree = parser.root()`;). Adicionalmente, ANTLR nos genera la signature de las clases visitor que se pueden ocupar para navegar sobre el árbol de parseo para armar un plan lógico inicial.

¹Nuestra implementación cuenta con un fragmento reducido de SQL. En nuestras consultas, solo podemos tener una **AND** de condiciones dentro de un **WHERE** y no soportamos consultas anidadas o corelacionadas.

2. Visitors y un primer plan lógico (carpeta `query_preprocessor`) Luego de generar el árbol de parseo, `query_preprocessor` recorre este árbol usando los distintos visitors para generar el plan lógico inicial (el mismo explicado en las clases cual consiste de una proyección, seguida por una selección y un producto cruz de las tablas). Durante este proceso la variable `current_logical_plan` de la clase `QueryPreprocessor` colecciona los elementos necesarios para armar el plan (variables por proyectar, condiciones, relaciones, atributos, alias para tablas (palabra `AS` en SQL), etc.). Los visitors son un patrón de diseño que nos permite navegar la estructura y cambiarnos de una clase a otra usando polimorfismo sin tener que re-implementar cada clase que vamos a ocupar. La estructura del plan lógico, dependiendo de si se trata de un plan de selección, proyección, etc. está descrita en la carpeta `logical_plan`. Las expresiones que pueden aparecer en un `WHERE` (por ejemplo `R.A = S.B` o `R.A > 7`) tienen su propia estructura y sus visitors son definidos de manera especial (subcarpeta `logical_plan/expr`). Si quieren empezar a entender cómo los visitors crean un plan lógico se les recomienda revisar el código.

(Si quieren estudiar sobre el visitor pattern/ANTLR por su propia cuenta, se recomienda el siguiente recurso:

<https://tomassetti.me/getting-started-antlr-cpp/>)

3. Reescritura de consultas (carpeta `query_rewriter`) Ahora que ya contamos con un primer plan lógico, lo optimizaremos un poco usando las reglas de reescritura de consultas. Para contar con un código acotado, en este laboratorio las únicas dos operaciones que se harán es agrupar las relaciones que hacen el join y hacer push de selecciones hacia las hojas del árbol lógico. Estas dos operaciones se hacen en `build_join.h` y `push_selection.h` respectivamente. Efectivamente, `build_join.h` va a definir cuales tablas hacen join entre ellas, y no ocupar el producto cartesiano en este caso, mientras `push_selection.h` pone las condiciones junto con los operadores/relaciones que los usan. Este tipo de optimizaciones estamos viendo en las clases actualmente y no las necesitan para las tareas de este laboratorio. Al terminar con este paso, contamos con un plan lógico más detallado que el original.

Optimizer, executor y el plan físico [informativo – punto de partida del Lab3]

Ahora que ya contamos con un plan lógico, lo vamos a convertir a un plan físico. Esto ocurre en el módulo `src/query/optimizer` en la clase `Optimizer`. El plan físico es simplemente un iterador sobre los resultados de nuestra consulta cual implementa la clase abstracta `QueryIter` (ver `query_iter.h` en la carpeta `src/query/executor`)².

El plan físico es construido por la clase `Optimizer` de nuevo usando un visitor. El iterador que define nuestro plan físico está compuesto de varios iteradores que implementan la clase abstracta `QueryIter`, uno para cada operador de álgebra relacional soportado por nuestro fragmento de SQL (para ver sus implementaciones; o sea, los operadores físicos que los definen, consulten `src/query/executor`). Estos `QueryIters` se componen siguiendo la estructura del plan lógico permitiendo una ejecución pipelined.

Nuestro `Optimizer` toma el plan lógico generado por el parser, y empieza visitar distintos operadores para generar sus respectivos `QueryIters`. Una tarea importante cual se realiza en este proceso es definir el orden de los joins y productos cruz. En nuestro caso, optimizer simplemente genera joins de una left-deep plan usando el orden de relaciones mencionado en la clausula `FROM` de nuestra consulta y nada más (i.e. no hay una estimación del costo). En un sistema más completo esta componente también haría una estimación de costo para elegir el mejor plan físico y el mejor orden de los joins.

Durante este proceso, cada operador entrega un `QueryIter` cual permite iterar sobre los resultados de este sub-árbol. Cómo explicaremos más abajo, un operador físico (e.g. la implementación de join) implementa un `QueryIter` cual entrega el siguiente resultado con la lógica de `next()` explicada en las clases. En este laboratorio ustedes implementarán esta interfaz para una clase especial de joins.

²También existen operadores del tipo `QueryAction` cuales se usan para las consultas del tipo `CREATE` o `INSERT`, pero en este laboratorio no exploraremos su implementación

Operadores físicos en IIC3413DB

Un operador físico en IIC3413DB es simplemente una implementación de la clase abstracta `QueryIter`, la cual requiere implementar un iterador lineal con la interfaz:

- `open()` – el método actualiza el estado inicial para luego poder conseguir la primera tupla del resultado llamando al `next()` por primera vez;
- `next()` – el método cual calcula cual es la siguiente tupla del resultado;
- `reset()` – el método cual reinicia el estado para poder ejecutar el operador de nuevo desde el inicio.

Igual cómo lo que vimos en la clase, el método `next()` retorna un `Record` de IIC3413DB. Para señalar que el siguiente record ya no existe (entregamos el último resultado), en método `next()` devuelve un `Record res` cual no es válido. Para ver si un record *no* es válido pueden usar el método `rec.invalid()`. En IIC3413DB usamos el record vacío para señalar que tenemos un record no válido, quiere decir un valor `Record()`. En breve, cuando llegamos al final de nuestros resultados, el `next()` debe siempre retornar `Record()`, y para validar si un hijo de nuestro operador llegó al final, podemos ocupar `auto childRec = child->next()` y luego `childRec.invalid()`.

Ayudantía: En la ayudantía del Lab3 explicando la estructura del código y cómo se evalúan los joins generales antes de explicar lo que hay que implementar en este lab.

Hash join en IIC3413DB

Para este laboratorio nuestra tarea será completar la implementación del operador físico de hash-join cual se encuentra en la carpeta `src/query/executor/hash_join`. Nuestra implementación permitira evaluar joins por un atributo, quiere decir, las consultas del estilo:

```
SELECT R.a
FROM R, S
WHERE R.b = S.b
```

Para implementar hash-join, nuestro punto de partida será la clase `HashJoin` (ver `hash_join.h`) la cual implementa el iterador `QueryIter` y contiene los siguientes elementos importantes (muy parecidos a los de la clase `Join` cual permite evaluar joins genéricos):

- `lhs` y `rhs` cuales son (punteros a) iteradores de las relaciones del lado izquierdo y derecho de nuestro join. Quiere decir que en el join $R \bowtie_{R.b=S.b} S$ del ejemplo anterior, nuestro plan recibe un iterador para R en la variable `lhs` y uno para S en `rhs`. Estos iteradores implementan la misma interfaz de `open()`, `next()` y `reset()` cómo todos los otros operadores físicos.
- `projected_lhs_columns` – el vector cual contiene las columnas (atributos) de la relación al lado izquierdo del join cuales se usaran en nuestro plan (y lo mismo para `rhs` en la variable `projected_rhs_columns`). Aquí es importante entender que se guardan solo los atributos necesarios para ejecutar el join *y el resto de la consulta* y no necesariamente todos los atributos de las tablas ocupadas. Por ejemplo, en la consulta de arriba las columnas del `lhs` (quiere decir de R) necesarias para ejecutar la consulta serán solo `a` y `b` (primero para `SELECT`, segundo para el join), y no todas las columnas de R . Esta optimización la explicamos en clases y se llama *selection pushdown*.
- `equalities` – un par de índices en `projected_lhs_columns` y `projected_rhs_columns` de columnas cuales hacen el join. En nuestro ejemplo estos serían `(1,0)` si tenemos `projected_lhs_columns = [a,b]` y `projected_rhs_columns = [b]`, dónde `[a,b]` indica la lista de dos valores.

Adicionalmente, la clase contiene los siguientes dos elementos cuales nos permitirán implementar nuestra propia tabla de hash:

- **htbl** cual es un puntero al objeto de la clase **HashTable**; y
- **hash_lhs** cual es un iterador sobre los resultados del probe en nuestra tabla de hash.

A continuación explicaremos cómo se implementa el **HashJoin** en **IIC3413DB**.

Linear hash table con buffer de tuplas

Para implementar hash-join primero implementaremos una tabla de hash cual ocupa un espacio del tamaño fijo en memoria principal. La clase **HashTable** en **src/query/executor/hash_join/hash_table.h** contiene la signatura de esta clase.

Dado que la tabla de hash será ocupada para almacenar records del lado izquierdo del join, la clase **HashTable** utiliza dos buffers de tamaño fijo alocados en memoria al momento de su construcción:

- **probe_table**: un directorio de **CAPACITY = 4096** entradas de tipo **TableEntry**. Cada entrada almacena tres campos: (i) el hash de la llave de join, (ii) su valor codificado (si el atributo de join es **INT** es el valor exacto y para **STR** son los primeros 8 bytes), y (iii) **head_offset**, un *offset* dentro de **record_buffer** que apunta al registro más recientemente insertado con esa llave, o **-1** si el slot está vacío. Las colisiones entre llaves distintas se resuelven con *linear probing* circular: si el slot calculado ya está ocupado por una llave distinta, se avanza al siguiente slot. El campo **head_offset** siendo **-1** significa un slot vacío.
- **record_buffer**: un buffer plano de **RECORD_BUFFER_SIZE = 1 MB** donde los registros de **lhs** se serializan de forma contigua. Cada registro va precedido por un campo de 8 bytes (**next**) que almacena el offset del registro anterior con la misma llave de join, o **-1** si es el primero de su cadena. Los registros con la misma llave quedan así enlazados en una lista simplemente ligada dentro del buffer.

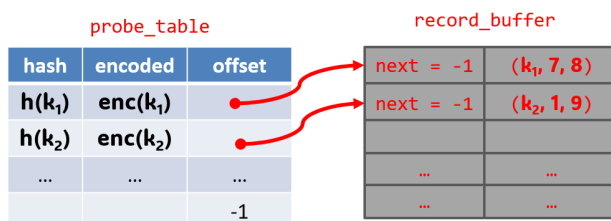


Figura 1: Probe table y record buffer

Para trabajar con los records, nuestra clase **HashTable** tiene dos campos importantes que le permiten identificar la llave indexada (el atributo cual hace el join) y el esquema de la tabla cuyas tuplas se procesan con el hash table. Estos se guardan en:

- **int column_idx** – índice de la columna cual hace el join;
- **std::vector<DataType> datatypes** – la lista con tipos de datos de la tupla (esto es igual a los tipos de **projected_lhs_columns** en la clase **HashJoin**).

Para insertar un objeto de tipo **Record** a **HashTable**, se ocupa la función:

```
bool try_insert_record(const Record& record)
```

La inserción (`try_insert_record`) funciona de la siguiente manera:

1. Primero se valida si `record_buffer` contiene el espacio suficiente para insertar `record`. El espacio ocupado para colocar el record se puede calcular usando `record.get_size_for_serialize() + sizeof(uint64_t)`. La primera parte de esta formula usa un metodo nuevo de la clase `Record` (cual ustedes implementaron en Lab1) y la segunda parte corresponde al `next` mencionado arriba.
2. Luego se calculan el `hash` del atributo de join, el `probe_slot` al cual va nuestro record, y `encoded_value` para el atributo del join.
3. Se busca el slot correspondiente en `probe_table` con linear probing (ver clase 8). Una diferencia aquí es que cada vez que pasamos por un slot cual ya contiene entrada *con el mismo has y el mismo encoded_value*, el nuevo registro se *antepone* a su lista enlazada almacenando el `head_offset` actual como el campo `next` del nuevo registro y actualizando `head_offset` al offset del nuevo registro en `record_buffer`. Si la llave es nueva y no se ha alcanzado el límite `MAX_ENTRIES = 3000`, se crea un nuevo slot en el directorio.
4. Si no hay espacio suficiente en `record_buffer` para el nuevo registro, `try_insert_record` retorna `false` sin modificar la tabla. En los dos caso el método devuelve `true`.

El *chaining* permite almacenar múltiples registros con la misma llave de join en el mismo slot del directorio, lo cual es necesario cuando varios registros de `lhs` comparten el valor del atributo de join. Sin encadenamiento, las tuplas con la llave repetida se agruparían cerca del mismo slot del `probe_table`, resultando en una tabla de hash lenta. La restricción `MAX_ENTRIES < CAPACITY` mantiene el factor de carga del `probe_table` por debajo de 0,75, limitando la degradación del linear probing.

Las siguientes figuras ilustran el ejemplo de insertar tres records con el esquema (STR,INT,INT) y el atributo de join siendo el primer atributo.

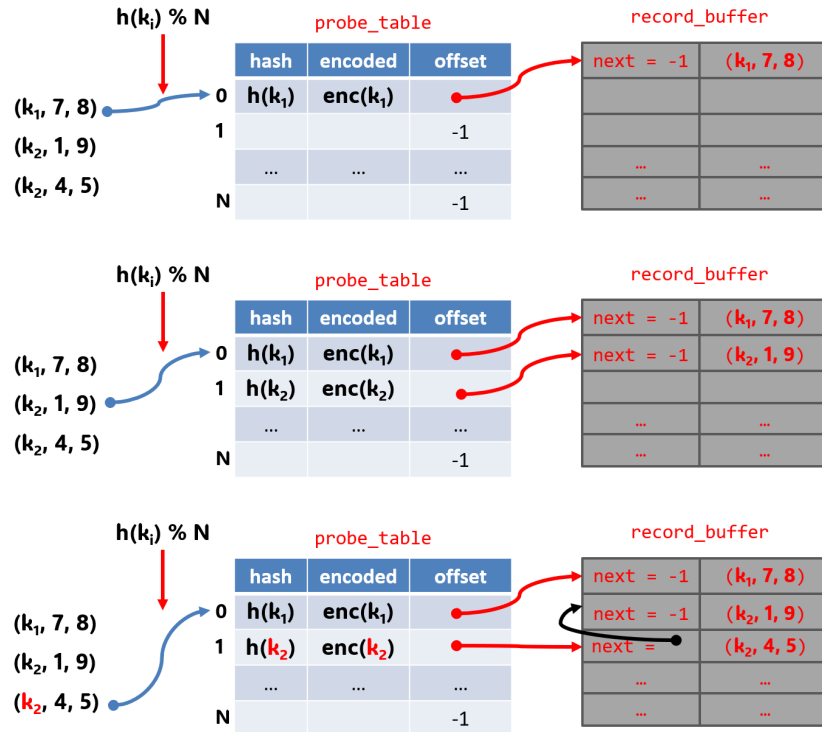


Figura 2: Insertando tres records a la tabla de hash

Probing en nuestra tabla de hash

La segunda operación fundamental para el funcionamiento de nuestra tabla de hash es poder buscar las tuplas que fueron ingresadas y hacen match con una llave en particular. El probing se implementa como un iterador (heredando de `RelationIter`). A grandes rasgos, la lógica de este iterador (dado una llave de búsqueda `key`) es la siguiente:

- `begin()` realiza linear probing en `probe_table` para encontrar el slot correspondiente a la llave buscada `key`, y guarda el `head_offset` de ese slot para poder acceder a la cadena de las tuplas con esta llave de búsqueda.
- `next()` deserializa el registro en la posición actual de `record_buffer`, lee el campo `next` para avanzar al siguiente nodo de la cadena, y retorna el registro deserializado. Cuando el campo `next` del `record_buffer` llega a `-1` (fin de la lista), `next()` retorna `Record()` indicando que no hay más coincidentes.
- `reset()` pone el iterador al mismo estado cómo después de `begin()`.

Idea de tener probe cómo un iterador surge de su uso en hash join, dónde un record del `rhs` puede hacer match con varios records del `lhs` cuales se guardan en una cadena en nuestro `HashTable`.

El siguiente ejemplo ilustra cómo `HashTableIter` recorre los registros con llave k_2 en la tabla construida anteriormente (último imagen en Figura 2):

- Tras `begin()`, el iterador consigue el offset de la tupla $(k_2, 4, 5)$ en el `record_buffer` buscando el `probe_table`.
- Primer `next()` retorna $(k_2, 4, 5)$ deserializandolo desde el `record_buffer` y consigue el offset apuntado por su `next` (a la posición de la tupla $(k_2, 1, 9)$).
- Segundo `next()` retorna $(k_2, 1, 9)$ y ve fin de la cadena.
- Cada subsiguiente `next()` retorna `Record()`.

Hash join en IIC3413DB

La implementación de `HashJoin` sigue el algoritmo clásico de hash join en dos fases: *build* y *probe*. En la fase de *build* se construye la tabla de hash a partir de los registros de la relación izquierda (`lhs`). En la fase de *probe* se itera sobre la relación derecha (`rhs`) y, para cada registro, se consulta la tabla de hash para obtener todos los registros de `lhs` que hacen match. Recuerden que aquí la tabla de hash devuelve un `HashTableIter` el cual nos permite iterar, una por una, sobre las tuplas del `lhs` cuales hacen match con la tupla del `rhs`. Los pares resultantes se emiten como tuplas del join.

Dado que `HashTable` tiene tamaño fijo (acotado por `RECORD_BUFFER_SIZE` y `MAX_ENTRIES`), es posible que `lhs` no quepa completamente en memoria. Para manejar este caso, el join se ejecuta en *bloques*: `lhs` se particiona implícitamente en bloques del tamaño que cabe en la tabla, y por cada bloque se realiza un ciclo completo de build seguido de un escaneo completo de `rhs`. Si `lhs` cabe completamente en la tabla, el join se completa en un único ciclo BUILD+PROBE.

Tareas para hacer en este laboratorio

Problema #1: `try_insert_record` [2 puntos]. Su primera tarea en este laboratorio es completar la implementación del método `try_insert_record` en `hash_table.cc` según la lógica explicada. Para hacer tests de su implementación les dejamos un método cual imprime el contenido de la tabla de hash y del record buffer. También, `src/bin/test_build_hash.cc` contiene una prueba con esto.

Hint: Cuando quieren escribir el offset de `next` en el record buffer, deben usar:

```
memcpy(record_buffer + offset1, &i, sizeof(int64_t))
```

Similarmente, para escribir un `Record` `rec` en el record buffer, deben utilizar:

```
rec.serialize(record_buffer + offset2)
```

Este último usa el nuevo método de la clase `Record` llamado `serialize` cual escribe el contenido del record en la posición de memoria apuntada por el parámetro recibido. Tengan cuidado de calcular correctamente los offsets cuando serializan una tupla según la lógica explicada (`next` y la tupla entera).

Problema #2: HashTableIter [1 punto]. En este ejercicio deben implementar `begin()`, `next()` y `reset()` del `HashTableIter` en el archivo `hash_table_iter.cc`. Para esto pueden hacer lo que quieran, pero el funcionamiento del iter debe ser cómo explicamos arriba. Tengan cuidado de que un `reset()` debe permitir iterar de nuevo sobre la cadena con el mismo `key`. En `src/bin/test_hash_iter.cc` le dejamos un test para validar su solución.

Hint: Los elementos de la clase `HashTableIter` incluyen los siguientes:

- `HashTable& htbl` – cual les permite acceder a la tabla de hash y ver sus tipos de datos/esquema.
- `Value key` – la llave de búsqueda; y `int column_idx` – la columna de `key`.

Para leer un record desde el `record_buffer`, pueden utilizar:

```
Record record = Record::deserialize(htbl.record_buffer + record_offset, htbl.datatypes);
```

Para leer el valor del `next` desde el `record_buffer`, pueden utilizar:

```
memcpy(&next, htbl.record_buffer + offset, sizeof(int64_t));
```

dónde `next` es definido con `uint64_t next`;

Problema #3: Hash Join [1 punto + 2 puntos]. En este problema se les pide completar la implementación del `next()` en la clase `HashJoin` en `hash_join.cc` según lo explicado arriba. Su solución se validará en tres fases:

1. Si no usan `htbl` en su solución y hacen match directamente con tuplas del `lhs` reciben 0 puntos [**sin posibilidad de reclamar**].
2. Si su solución funciona solo con una ronda de build (i.e. asumen que todas las tuplas del `lhs` caben en nuestro `HashTable` de tamaño fijo) reciben +1 punto.
3. Si su solución funciona para cualquier tamaño de `lhs` reciben +2 puntos.

Los últimos dos puntos se revisarán por separado. Test que les preparamos en `bin/src/hash_join_test.cc` considera que `lhs` entero cabe en la tabla. Para el último punto ustedes deben armar sus propios tests.

Hint: Un posible método para solucionar este problema es pensar que su `next()` puede estar en uno de los siguiente tres estados y actual dependiente de esto:

- `BUILD` – hay que armar la tabla de hash con las tuplas que siguen en `lhs`.
- `PROBE` – estamos iterando sobre `rhs` y buscando resultados en la tabla existente.

- DONE – ya llegamos al final de ambos lhs y rhs.

Problema Bonus: Index nested loops join [+1 punto en cualquier lab]. En este bonus su optimizador debe poder detectar si existe un hash index sobre el atributo del join (en el caso que se usa solo un atributo para este join) y en este caso ejecutar la implementación del join ocupando el índice para la relación indexada. Esto quiere decir que deben tener otro operador físico para joins, el cual se usa solamente en el caso de poder hacer el join ocupando el índice para la tabla del lado derecho. Efectivamente, en un join $R \bowtie_{R.A=S.B} S$, en el caso de contar con un índice sobre B en S , se debe ocupar este operador. La implementación del hash join se les pasará junto con este lab en el caso de no resolver Lab 2. (Su solución se revisará a mano para asegurarnos que están ejecutando el join usando índices. Cómo se trata de bonus no se entregará información extra sobre cómo solucionar/testear este problema.)

Bug hunt [+0.2 puntos en la nota del lab]. Si encuentren un bug no-trivial en nuestra implementación y lo reportan a través de issues de *gitHub* se les asignará una décima a la nota del laboratorio. Sí, pueden contar con una nota mayor que 7 en este caso (e.g. 7.2) y esta nota se considerará para el promedio de los labs.

Código

El código base está disponible en [1] en la rama **Lab3**. Asegúrense que están usando la última versión del sistema! Como parte del código, también publicaremos la solución de tarea 2 (cual sirve solo para el bonus). El código también contiene un cliente que les permite ejecutar las consultas sin tener que modificar un `.cc`. Este también permite imprimir planes (lógicos y físicos). Instrucciones están en el readme del código. También hay un par de tests que pueden ejecutar cómo siempre.

Evaluación y entrega

El día límite para la entrega de esta tarea será el viernes 15 de mayo a las 23:59 horas. Para ello se utilizará el formulario de canvas. *Su entrega debe contener todo el código de la carpeta **src** y nada más, y debe compilar cuando se les agregan otras carpetas en el repositorio. El código de la carpeta **src** debe ser entregado cómo un archivo **.zip**.* Por último, la evaluación será en base a varios test que su solución debe responder.

Ayudantía y preguntas

Ayudantía: viernes 8 de Mayo. Preguntas: issues de github.

Referencias

[1] IIC3413DB. <https://github.com/IIC3413/2026-1>.

[2] Terence Parr. ANTLR. <https://www.antlr.org/>.